

Language Server Protocol para YAP

Programação em Lógica
CC3012

2026

Docente: Vitor Santos Costa

André Mendes up202307449
Eurico Magalhães up200701520

Conteúdos

Motivação	4
Language Server Protocol	4
pygls	5
Visual Studio Code ou Emacs	6
Configuração do sistema	7
Ambiente Virtual Python	8
YAP	8
LSP4YAP	9
Emacs	9
Utilizar lsp4yap	9
Funcionamento	10
lsp4yap	10
YAP	12
validate_text/3	12
term_expansion/2	14
analyze/5	14
rule/5	15
body/5	16
directive/4	17
portray_message/2 e q_msg/3	18
Problemas	20
Alternativas	22
Expressões Regulares	22
Gramática Independente de Contexto	24
Global	31
list_calls/1	33
list_by_caller/2	33
list_by_called/2	33
list_by_arity/2	33
entry_points/1	33
undefined_calls/1	34
edge/2	34
connected/3	34
reachable_from/2	35

cycles/1	35
Conclusão	36

Motivação

Um dos possíveis trabalhos apresentados pelo professor foi o desenvolver uma solução para o seguinte problema: os utilizadores de editores de texto e IDE's esperam que estes programas possuam certas funcionalidades, como coloração de texto consoante a sintaxe da linguagem de programação, verificação e validação dessa sintaxe, procura de símbolos e definições, entre outras.

O compilador de Prolog YAP ainda não tem uma integração com possíveis soluções para este problema. Assim, este trabalho pretende investigar e implementar uma solução, usando o Language Server Protocol.

Language Server Protocol

O Language Server Protocol¹(LSP) é um protocolo de comunicação entre editores de texto e IDE's, e um servidor de uma linguagem de programação, que providencia ao editor as regras e instruções necessárias à implementação de funcionalidades de formatação e uso dessa linguagem no contexto do editor ou IDE.

Desenvolvido pela Microsoft, e *open source*, é um protocolo usado no Visual Studio Code²(VSCode) e, como este editor é bastante usado, atualmente, pelos alunos de Ciência de Computadores, o professor decidiu trabalhar neste contexto.

A motivação por trás do LSP, como explica a Microsoft, é a seguinte: em vez de cada editor de texto ter que implementar todas as linguagens de programação que pretende suportar, porque não ter um protocolo de comunicação que, quando implementado uma vez em cada editor, permite reconhecer todas as linguagens que implementem, por sua vez, esse protocolo.

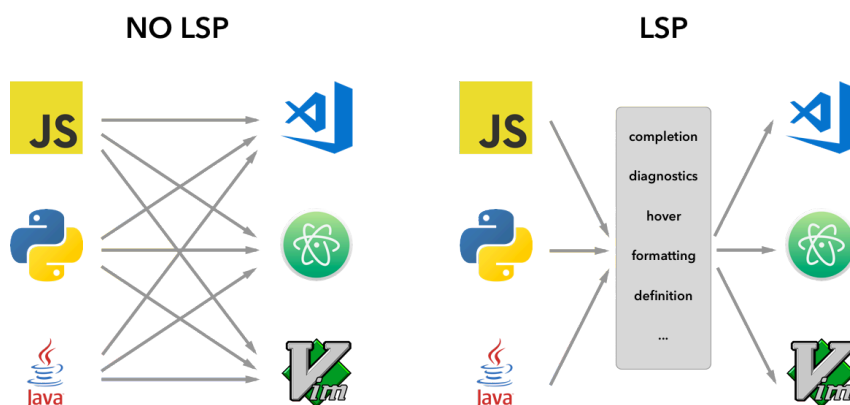


Figura 2: Motivação do LSP (fonte: site da API do VSCode²)

¹<https://microsoft.github.io/language-server-protocol/>

²<https://code.visualstudio.com/api/language-extensions/language-server-extension-guide>

Este protocolo implementa mensagens codificadas em JSON RPC³ que são trocadas entre cliente e servidor.

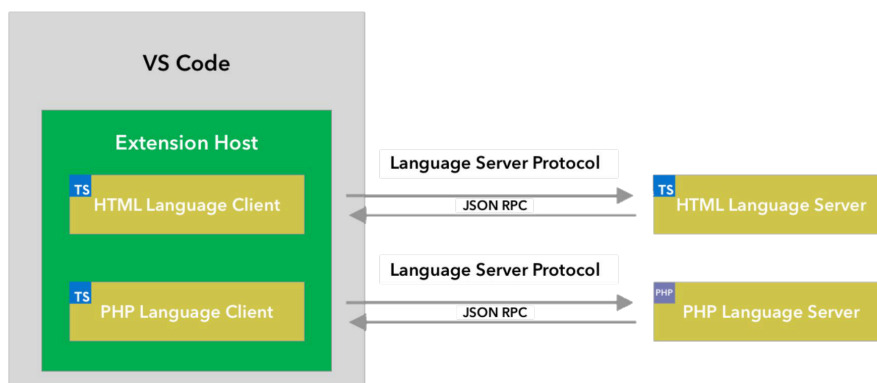


Figura 3: Funcionamento do LSP (fonte: site da API do VSCode²)

Como este protocolo se tornou popular, foram desenvolvidas várias ferramentas que permitem mais facilmente implementar um cliente/servidor LSP, evitando assim termos que, por exemplo, implementar todas as mensagens JSON.

Uma destas ferramentas é o pygls⁴.

pygls

O pygls é uma “implementação genérica do LSP escrita em Python”⁴. Usando o pygls deve ser possível escrever um servidor LSP em Python, de uma maneira relativamente simples.

Por exemplo, o código seguinte recebe um documento e, em cada linha que acabe em “hello.”, retorna ao editor de texto a hipótese de autocompletar com “world” ou “friend”, sempre que é introduzido um “.”.

³<https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>

⁴<https://pygls.readthedocs.io/en/latest/>

Exemplo de um lsp.py

```

1 from pygls.lsp.server import LanguageServer
2 from lsprotocol import types
3
4 server = LanguageServer("example-server", "v0.1")
5
6
7 @server.feature(
8     types.TEXT_DOCUMENT_COMPLETION,
9     types.CompletionOptions(trigger_characters=["."]),
10    0 autocompletar só aparece quando se insere um "."
11 )
12 def completions(params: types.CompletionParams):
13     document =
14     server.workspace.get_text_document(params.text_document.uri)
15     current_line = document.lines[params.position.line].strip()
16
17     if not current_line.endswith("hello."):
18         return []
19
20     return [
21         types.CompletionItem(label="world"),
22         types.CompletionItem(label="friend"),
23         Itens que aparecem no autocompletar do editor de texto
24     ]
25
26 if __name__ == "__main__":
27     server.start_io()

```

O cliente dependerá, claro, do editor de texto/IDE. É aqui que surge o primeiro impedimento ao objetivo do trabalho.

Visual Studio Code ou Emacs

O VSCode⁵, nas palavras do professor, pode ser “bastante burocrático” na implementação de um cliente LSP. De facto, esse cliente teria de ser um projecto implementado em TypeScript, e o *debugging* envolve também muita burocracia, especialmente quando comparado com a alternativa: Emacs.

O Emacs⁶ é muito personalizável e, crucialmente, tem uma extensão que permite trabalhar com LSP, Eglot⁷. É relativamente fácil fazer com que esta extensão lance o nosso servidor,

⁵<https://code.visualstudio.com/>

⁶<https://www.gnu.org/software/emacs/>

⁷<https://joaotavora.github.io/eglot/>

como veremos no próximo capítulo. Permite também monitorizar as mensagens enviadas entre Emacs e o servidor de uma maneira simples.

Configuração do sistema

No seu presente estado, este projecto precisa de três componentes:

- Um editor de texto/IDE. Usaremos Emacs, pelas razões mencionadas acima. Este editor de texto tem um cliente de LSP (Eglot);
- Um servidor de LSP. O professor providenciou o repositório `lsp4yap`, tanto no GitHub⁸ como no Codeberg⁹. Crucialmente, este repositório contém um ficheiro `yap.py`, que é o ponto de entrada do servidor. Este ficheiro, atualmente, recebe mensagens do editor (cliente LSP) e usa o próprio YAP para processar e assinalar com erros o texto do documento, enviando mensagens ao servidor `yap.py`, que depois as reenvia ao cliente;
- Um compilador de Prolog. Como vimos na entrada anterior, o nosso servidor precisa do YAP para funcionar. E obviamente faz sentido que, se estamos a escrever código em Prolog, temos um compilador dessa linguagem no mesmo sistema.

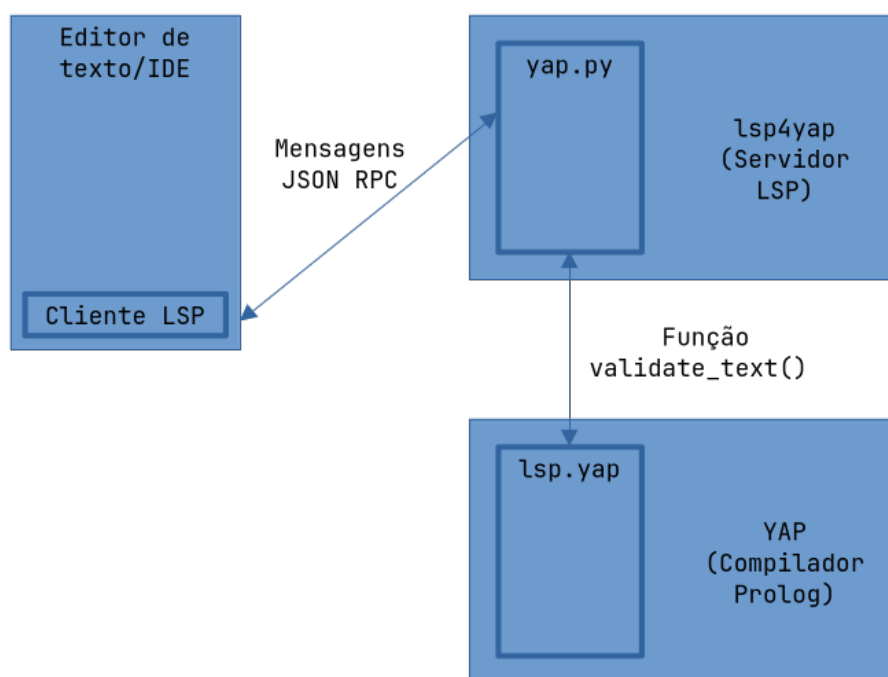


Figura 4: Visão global do projeto

Exploraremos os vários componentes com mais detalhe, mas primeiro preparemos o sistema.

Alguns caminhos importantes:

- `$ENV` é o caminho para um ambiente virtual Python, por exemplo `~/env`;

⁸<https://github.com/vscosta/lsp4yap>

⁹<https://codeberg.org/vscosta/lsp4yap>

- \$LSP4YAP é o caminho para o clone do repositório lsp4yap^{8,9};
- \$YAP é o caminho para o clone do repositório yap^{10,11};

Procedemos da seguinte maneira:

Ambiente Virtual Python

1. Criar um ambiente virtual Python. Navegar para o local pretendido (por exemplo ~/) e executar:

bash

```
python3 -m venv env
```

2. Este ambiente tem de ser ativado, para podermos usar os comandos python e pip a partir de \$ENV, e não do sistema. Se a shell usada for fish, o ficheiro a tocar é ligeiramente diferente:

bash

```
source $ENV/bin/activate
```

fish

```
source $ENV/bin/activate.fish
```

Para verificar se estamos a usar o ambiente virtual, executar os comandos seguintes e confirmar que o caminho devolvido é em \$ENV:

bash

```
which python
which pip
```

3. Podemos agora instalar as dependências Python:

bash

```
pip install lsprotocol pygls numpy
```

YAP

4. Clonar o repositório yap^{10,11} para \$YAP:

bash

```
git clone <yap_URL>
```

5. No diretório \$YAP, criar um diretório Build e navegar para aí:

bash

```
mkdir Build
cd Build
```

6. Compilar e instalar YAP, a partir de Build:

bash

```
cmake ../
make
```

¹⁰<https://github.com/vscosta/yap>

¹¹<https://codeberg.org/vscosta/yap>


```
sudo make install
```

7. Navegar para \$YAP e instalar o pacote Python yap4py no ambiente virtual (confirmar que está ativado, ponto 2):

bash

```
pip install packages/python/yap4py
```

LSP4YAP

8. Clonar o repositório lsp4yap^{8,9} para \$LSP4YAP:

bash

```
git clone <lsp4yap_URL>
```

Emacs

9. Instalar Emacs;
10. Configurar Emacs: no ficheiro de configuração, colocar:

lisp

```
(with-eval-after-load 'eglot
  (add-to-list 'eglot-server-programs
    '(prolog-mode . (" $ENV/bin/python" "$LSP4YAP/yap.py"))))

(add-hook 'prolog-mode-hook 'eglot-ensure)

(setq auto-mode-alist (append '(("\\.pl\\\\" . prolog-mode)
                                ("\\.yap\\\\" . prolog-mode))
                              auto-mode-alist))
```

Utilizar lsp4yap

11. No Emacs, abrir um ficheiro de Prolog e fazer M-x eglot (Alt+X, escrever eglot, pressionar Enter)

Funcionamento

lsp4yap

Atentemos agora com algum cuidado ao diretório `lsp4yap`.

O ficheiro `yap.py` é o ponto de entrada do nosso servidor. De notar que, no ficheiro `packages.json`, alguma variáveis devem ser definidas. Por exemplo:

json

packages.json

```
1 "configuration": [  
2   {  
3     "type": "object",  
4     "title": "Server Configuration",  
5     "properties": {  
6       "pygls.server.launchScript": {  
7         "scope": "resource",  
8         "type": "string",  
9         "default": "examples/servers/yap.py",  
10        "description": "The python script to run when launching the  
server.",  
11        "markdownDescription": "The python script to run when launching  
the server.\n Relative to #pygls.server.cwd#"  
12      }  
13    }  
14  }  
15 ]
```

Define que o servidor `pygls` deve correr `yap.py` quando inicia.

É importante recordar que, na configuração do Emacs, nós definimos que a extensão Eglot deve correr o ficheiro `yap.py` quando entra no modo Prolog, “ignorando” assim estas configurações. Esta é mais uma razão para ter escolhido Emacs, a facilidade de ligar o editor de texto com o ficheiro que especificamos diretamente. Os ficheiros de configuração envolventes que o `pygls` providencia são úteis quando quisermos adaptar este projeto para Visual Studio Code.

No ficheiro yap.py, uma função importante é validate():

python

yap.py

```
1 def validate(self, document: TextDocument):
2     """Validates prolog file."""
3
4     diagnostics = []
5     Este array de Diagnostics é preenchido com os vários erros
6     uri = document.uri
7     print(" Master!!!!!!!!!!!!!!", file = sys.stderr)
8     data = engine.fun(validate_text(uri,document.source))
9     A função validate_text() interage com o YAP, processando assim o
10    documento num contexto de prolog, e recebendo como retorno erros, que são
11    guardados em data.
12    print(" Master:vdone")
13
14    errs = data
15
16    if errs:
17        Esse array de erros é depois processado aqui, num contexto de pygls.
18        Range, Position, Diagnostic, entre outras, são construções que o pygls
19        usa para definir as mensagens JSON RPC trocadas entre cliente e servidor
20        LSP.
21
22        for (sev,msg,i0,j0,i1,j1) in errs:
23
24            if sev == "warning":
25                sev=types.DiagnosticSeverity.Warning
26            else:
27                sev=types.DiagnosticSeverity.Error
28
29            location=types.Range(
30                start=types.Position(line=i0-1, character=j0-1),
31                end=types.Position(line=i1-1, character= j1-1)
32            )
33
34            diagnostics.append(
35                types.Diagnostic(
36                    message = msg,
37                    severity=types.DiagnosticSeverity.Warning,
38                    range = location
39                )
40            )
41
42    Aqui construímos os vários Diagnostic, para cada erro reportado. Usamos
43    os valores retornados pela função YAP validate_text() (sev, msg, e
44    valores de linha e coluna).
45
46    )
47
48    return document.version,diagnostics
```

YAP

Ainda no ficheiro `yap.py`, a função `validate_text()` interage com `engine`, que é inicializado da seguinte maneira:

python

yap.py

```
1 from yap4py.yapi import Engine, EngineArgs
2
3 # [...] #
4
5 def start_yap():
6     eargs.jupyter = True
7     try:
8         engine = Engine( eargs)
9         engine.load_library("lsp")
10    except Exception:
11        print("bad load")
12        engine = None
13    return engine
```

`yap4py.yapi`, em `$YAP/packages/python`, funciona como um interpretador de Prolog dentro do YAP, capaz de receber instruções em Python e corrê-las em Prolog.

Desta maneira, usamos o compilador YAP como um interpretador de texto Prolog que, em vez de converter esse texto para código máquina (como o faria em “modo de compilador”), reporta apenas erros sintáticos e informa-nos acerca de tabelas de símbolos, as funcionalidades que requeremos para implementar um LSP.

validate_text/3

Assim, voltando a `yap.py`, a linha:

python

```
data = engine.fun(validate_text(uri,document.source))
```

É processada pelo `engine`, que é um “interpretador”, e corre a função `validate_text(uri,document.source)`. Consegue fazê-lo porque importa a biblioteca `lsp`, que é definida no ficheiro `lsp.yap`, também em `$YAP/packages/python`. Dentro deste ficheiro está definido o predicado correspondente, `validate_text/3`:

lsp.yap

```
1 validate_text(URI,S,Ts) :-
2
3     writeln(user_error, URI),
4
5     string_concat("file://", FileAsS, URI),
6     atom_string(FileAsS, File),
7
8     retractall(def(_,_,_,_,File,_)),
9     retractall(use(_,_,_,_,_,_,File,_)),
10    retractall(dec(_,_,_,_,File,_)),
11    Apaga dados anteriores.
12
13    open(string(S),read,Stream),
14    current_source_module(DefaultModule,user),
15    Abre o texto como uma Stream.
16
17    assert(lsp(on)),
18    Liga o "modo" LSP.
19
20    load_files(File,[stream(Stream)]),
21    current_source_module(_,DefaultModule),
22    Carrega e processa a Stream de texto.
23
24    retractall(lsp(_)),
25    Desliga o "modo" LSP.
26
27    findall(T,retract(m(T)),Ts).
28    Recolhe todos os erros.
```

term_expansion/2

Em Prolog, existem predicados chamados *hooks* que são corridos automaticamente pelo YAP¹². Assim, a linha `load_files(File,[stream(Stream)])` ativa o hook `term_expansion/2`, que é (re)definido neste ficheiro:

prolog

lsp.yap

```
1 user:term_expansion(G, GF) :-
2
3     lsp(on),
4     Só corre se estivermos no "modo" LSP
5     prolog_load_context(file, F ),
6     prolog_load_context(term_position, Pos ),
7     current_source_module(M, M ),
8     arg(2, Pos, Line),
9
10    analyse(G,Line,F,M,GF),
11    Analisa o termo Prolog
12    !.
```

analyze/5

`analyse/5` descobre que tipo de termo Prolog estamos a tratar:

prolog

lsp.yap

```
1 analyse((:- G), L, F, M, (:- true)) :-
2     !,
3     directive(G, L, F, M).
4     Cláusulas que só têm corpo.
5 analyse((:- _G), _L, _F, _M, (:- true)) :-
6     !.
7     Outras cláusulas só com corpo.
8 analyse(Cl, L, F, M, Na) :-
9     rule(Cl, L, F, M, Na).
10    Cláusulas de Factos e Regras, a maioria do casos.
11 analyse(_G, _L, _F, _M, Na) :-
12     strip_module(G,_MH,A),
13     functor(A,Na,_Ar).
14    0 resto que não for apanhado pelos casos anteriores.
```

¹²<https://www.swi-prolog.org/pldoc/man?section=hooks>

rule/5

O caso mais comum é uma Cláusula de Factos ou Regras. Relembremos que Factos são do tipo:

```
p(a).
```

prolog

E Regras são:

```
q(a) :- p(a).
```

prolog

Portanto, para explorar estes casos, usamos o predicado rule/5:

prolog

lsp.yap

```
1 rule((A0:- B),Line,File,Module,Na) :-
2
3     strip_module(Module:A0,MH,A),
4     functor(A,Na,Ar),
5     Extrair nome do predicado, o seu módulo e argumentos.
6     (
7         def(Na,Ar,MH,_,_,predicate)
8         ->
9         true
10        Verificar se a definição desse predicado já existe. Se existir, não fazer nada.
11        ;
12        assert(def(Na,Ar,MH,Line,File,predicate))
13        Se não existir, guardar na base de dados.
14        ),
15        body(B,Line,File,Module,MH:Na/Ar).
16    Analisar o Corpo da Regra.
```

Aqui encontramos o que pensamos ser um *bug*. Repare-se que ao fazer *pattern matching* no predicado analyze/5, uma cláusula do tipo Facto ou Regra seria apanhada pela terceira regra de analyze/5.

No entanto, essa regra chama o predicado rule/5. Este predicado só faz *pattern matching* com Regras (A0 :- B). Assim, Factos não são guardados na base de dados que depois será usada (eventualmente) para funções do LSP como *go-to-definitions*, análise semântica, entre outras.

Uma possível solução seria acrescentar o código seguinte ao predicado rule/5:

prolog

lsp.yap

```
1 rule(A0, Line, File, Module, Na) :-
2     strip_module(Module:A0, MH, A),
3     functor(A, Na, Ar),
4     (
5         def(Na, Ar, MH, _, _, predicate)
6     ->
7         true
8     ;
9         assert(def(Na, Ar, MH, Line, File, predicate))
10    ).
```

body/5

O corpo de uma regra é tratado pelo predicado body/5:

prolog

lsp.yap

```
1 body(A,_L,_F,_M,_) :-
2     var(A),
3     !.
4     Variáveis são ignoradas.
5 body(M:A,L,F,_M,P0) :-
6     !,
7     body(A,L,F,M,P0).
8
9 body((A,B),L,F,M,P0) :-
10    !,
11    body(A,L,F,M,P0),
12    body(B,L,F,M,P0).
13
14 body((A;B),L,F,M,P0) :-
15    !,
16    body(A,L,F,M,P0),
17    body(B,L,F,M,P0).
18
19 body((A->B),L,F,M,P0) :-
20    !,
21    body(A,L,F,M,P0),
22    body(B,L,F,M,P0).
23
24 body(A,L,F,M,M0:Na0/Ar0) :-
25     functor(A,NA,Ar),
26     assert(use(NA,Ar,M,Na0,Ar0,M0,L,F,predicate)).
27
28 Caso Base para o qual as outras regras convergem. Guardamos os vários
29 predicados do corpo na base de dados.
```


directive/4

Ainda a partir do predicado `analyze/5`, se encontrarmos uma directiva, ela é processada pelo predicado `directive/4`:

prolog

lsp.yap

```
1 directive(module(M,Ls),L,F,M0) :-
2     assert(def('',0,M,L,F,module)),
3     assert(use('',0,M,'',0,M0,L,F,module)),
4     current_source_module(M0,M),
5     maplist(mod(L,F,M),Ls).
Uma directiva do tipo module é guardada na base de dados, incluindo os seus
export.
6
7 directive(op(A,B,C), _Line,_File,M) :-
8     op(A,B,M:C).
9
10 directive(use_module(_A), _Line,_File,_M) :-
11     !.
12 directive(use_module(_A,_B), _Line,_File,_M) :-
13     !.
14 directive(load_files(_A,_B), _Line,_File,_M) :-
15     !.
16 directive(ensure_loaded(_A), _Line,_File,_M) :-
17     !.
18 directive(consult(_A), _Line,_File,_M) :-
19     !.
20 directive(reconsult(_A), _Line,_File,_M) :-
21     !.
22 directive(compile(_A), _Line,_File,_M) :-
23     !.
24 directive(use_module(_A,_B,_C), _Line,_File,_M) :-
25     !.
26
27 directive((A,B), Line,File,M) :-
28     !,
29     directive(A, Line,File,M),
30     directive(B, Line,File,M).
```

portray_message/2 e q_msg/3

O predicado seguinte, que também é um hook, corre quando o YAP deteta um erro¹³. Neste caso, term_expansion/2 não é chamado, mas sim este predicado:

prolog

lsp.yap

```
1 user:portray_message(A,B):-
2     lsp(on),
3     !,
4     writeln(user_error,B),
5     (
6         q_msg(A,B,T),
7         Converte a mensagem num tuplo
8         A \= "ignored"
9         ->
10        assert(m(T))
11        Guarda esse tuplo na base de dados
12        ;
13        true
14    ).
```

q_msg/3 filtra os vários tipos de mensagens geradas pelo YAP, e devolve as que são úteis ao LSP em tuplos t(Severity, Message, StartLine, StartColumn, EndLine, EndColumn).
prolog

lsp.yap

```
1 q_msg(informational, _, _) :-
2     !,
3     fail.
4
5 q_msg(help, _, _) :-
6     !,
7     fail.
8
9 Mensagens informacionais ou de ajuda são ignoradas.
10
11 q_msg(warning, error(style_check(singletons,
12 [VName,Line,Column,_F0],_),_Desc),t("warning",S, Line,Column,
13 Line,EndCol)) :-
14     !,
15     format(string(S), 'singleton variable ~s.~n ', [VName]),
16     atom_length(VName, Len),
17     EndCol is Column+Len.
18
19 Um aviso que reporta erros de singletons é formatado num tuplo de tipo
20 "warning", com a string definida aqui, e a sua localização em linhas e
21 colunas.
```

¹³<https://sicstus.sics.se/sicstus/docs/3.12.10/html/sicstus/Message-Handling-Predicates.html>

```

15 q_msg(warning, error(style_check(multiple,[F0|L],I ),_Desc ),
    t("warning",S, L,Column,L,EndCol)) :-
16     !,
17     Column=1,
18     format(string(S), '~w previously defined at ~s.~n',[I,F0]),
19     I = Name/_,
20     atom_length(Name, Len),
21     EndCol is Column+Len.
    0 mesmo predicado definido múltiplas vezes.
22
23 q_msg(warning, error(style_check(discontiguous,_,_I ), _Desc),
    t("warning",S, L,Column, L, EndCol)) :-
24     !,
25     Column=1,
26     format(string(S), 'discontiguous definition for ~w.~n',[I]),
27     I = Name/_,
28     atom_length(Name, Len),
29     EndCol is Column+Len,
30     S = "discontiguous.~n".
    Cláusulas do mesmo predicado que não estão agrupadas.
31
32 q_msg(_error, error(syntax_error(_Msg), Desc), t("error","syntax error",
    L,LPos,L,LP1)) :-
33     exception_property(parserLine, Desc, L),
34     exception_property(parserLinePos, Desc, LPos),
35     exception_property(parserSize, Desc, Size),
36     !,
37     LP1 is LPos+Size.
    Erros sintáticos.
38
39 q_msg(_,Error,t("ignored",Msg,1,0,2,0)) :-
40     term_to_string(Error,Msg).
    Quaisquer outros erros.

```

Problemas

O projeto encontra-se ainda em fase de desenvolvimento, tendo sido encontrados vários problemas ao longo do semestre.

Em primeiro lugar, e já mencionado acima, a intenção original era desenvolver uma extensão para o Visual Studio Code. No entanto, a excessiva burocracia e complexidade do VSCode levaram-nos, com a sugestão do professor, a desenvolver primeiro a parte do servidor LSP, e para isso usámos o Emacs, para ter o mínimo de fricção da parte do cliente/ editor de texto.

Porém, também na parte do servidor encontrámos problemas que não conseguimos resolver dentro do prazo.

O primeiro destes, que não deve ser menosprezado, é o entender o projeto em si. Este é o nosso primeiro contacto com uma linguagem lógica como o Prolog. Apesar de sermos familiares com linguagens funcionais, nós preferimos linguagens imperativas (por exemplo, no semestre passado, na Unidade Curricular de Compiladores, quando enfrentados com a escolha de escrever um compilador em Haskell ou C, escolhemos C). Assim, saltar de cabeça para um projeto tão complexo como este requereu estudar o código linha a linha, antes de tentar sequer implementar alguma coisa.

Tivemos também vários problemas com a configuração do projeto em várias máquinas. Por exemplo, o Eurico continua com erros no seu desktop:

```
1 [stderr] Traceback (most recent call last):
2 [stderr]   File "/home/eurico/Cloned-Repositories/Codeberg/lsp4yap/
  yap.py", line 41, in <module>
3 [stderr]     from yap4py.yapi import Engine, EngineArgs
4 [stderr]   File "/home/eurico/Projects/env/lib/python3.14/site-packages/
  yap4py/yapi.py", line 25, in <module>
5 [stderr]     from yap4py.queries import top_query
6 [stderr]   File "/home/eurico/Projects/env/lib/python3.14/site-packages/
  yap4py/queries.py", line 10, in <module>
7 [stderr]     from yap4py.yap import YAPQuery
8 [stderr] ModuleNotFoundError: No module named 'yap4py.yap'
9 [jsonrpc] D[09:21:32.141] Connection state change: `exited abnormally
  with code 1
```

Estes erros apontam para o módulo yap4py, do repositório yap, que o Eurico diz ter instalado correctamente, seguindo o guia de configuração apresentado neste relatório mais acima, com várias variações e a partir de vários repositórios de yap e lsp4yap.

Enquanto que no seu portátil, o Eurico reporta outro erro, este muito mais verboso, com linhas que apontam para, por exemplo:

```
1 /usr/local/share/Yap/yapi.yap:18:33: error: stream(0,pipe(9)):0:0: error
  executing prolog:throw/1
2 [...]
3 %%% domain error: consulted_at(_119) does not belong to domain
  implemented_option.
4 [...]
5 % info: yapi:load_files(library(maplist),
  [if(not_loaded),must_be_module(true),consulted_at(_119)])
```

Seria imprático colocar no relatório as centenas de linhas deste erro, mas o ponto é o seguinte: este projeto encontrou muitos erros de configuração e comunicação entre os vários ficheiros e programas que o compõem. O que é compreensível e até expectável, dada a dimensão e complexidade do projeto.

Alternativas

Seguem-se algumas alternativas que explorámos ao longo do trabalho, por vários motivos, o principal dos quais a simples exploração académica.

Expressões Regulares

Num primeiro encontro com o pygls, a maioria dos exemplos fornecidos na documentação envolvem alterar apenas o ficheiro principal Python (no nosso caso, `yap.py`). Deste modo, este ficheiro não dependeria de nada, para além das dependências intrínsecas ao pygls.

Usando o exemplo de Publish Diagnostics da documentação do pygls¹⁴ como base, é relativamente simples construir o seguinte ficheiro, que reconhece apenas factos e regras de prolog:

python

regex-linebyline.py

```
1 import logging
2 import re
3
4 from lsprotocol import types
5 from pygls.cli import start_server
6 from pygls.lsp.server import LanguageServer
7 from pygls.workspace import TextDocument
8
9 FACT = re.compile(r"^s*([a-z]\w*)\(((\s*\w+\s*,*\s*)+)\))\s*\.$")
10 Facto: "p(a)."
11
12 RULE = re.compile(r"^s*([a-z]\w*)\(((\s*\w+\s*,*\s*)+)\))\s*:\s*\s*\s*([a-z]\w*)\(((\s*\w+\s*,*\s*)+)\))\s*[\s*,\s*;\s*]+\s*\.$")
13 Regra: "q(a) :- p(a)."
14
15
16 class PublishDiagnosticServer(LanguageServer):
17     """Language server demonstrating "push-model" diagnostics."""
18
19     def __init__(self, *args, **kwargs):
20         super().__init__(*args, **kwargs)
21         self.diagnostics = {}
22
23     def parse(self, document: TextDocument):
24         diagnostics = []
25
26         for idx, line in enumerate(document.lines):
27             fact = FACT.match(line)
28             rule = RULE.match(line)
29
30             Em cada linha do documento, verificar se é um facto ou uma regra.
31
32             if fact is None and rule is None:
```

¹⁴<https://pygls.readthedocs.io/en/latest/servers/examples/publish-diagnostics.html>

```

28         message = "Not a fact or rule"
29         severity = types.DiagnosticSeverity.Error
30
31         diagnostics.append(
32             types.Diagnostic(
33                 message=message,
34                 severity=severity,
35                 range=types.Range(
36                     start=types.Position(line=idx, character=0),
37                     end=types.Position(line=idx,
character=len(line) - 1),
38                 ),
39             )
40         )

```

Se não for um facto ou uma regra, marcar toda a linha como um erro.

```

41
42         self.diagnostics[document.uri] = (document.version, diagnostics)
43         # logging.info("%s", self.diagnostics)
44
45
46     server = PublishDiagnosticServer("diagnostic-server", "v1")
47
48
49     @server.feature(types.TEXT_DOCUMENT_DID_OPEN)
50     def did_open(ls: PublishDiagnosticServer, params:
types.DidOpenTextDocumentParams):
51         """Parse each document when it is opened"""
52         doc = ls.workspace.get_text_document(params.text_document.uri)
53         ls.parse(doc)
54
55         for uri, (version, diagnostics) in ls.diagnostics.items():
56             ls.text_document_publish_diagnostics(
57                 types.PublishDiagnosticsParams(
58                     uri=uri,
59                     version=version,
60                     diagnostics=diagnostics,
61                 )
62             )
63
64
65     @server.feature(types.TEXT_DOCUMENT_DID_CHANGE)
66     def did_change(ls: PublishDiagnosticServer, params:
types.DidOpenTextDocumentParams):
67         """Parse each document when it is changed"""
68         doc = ls.workspace.get_text_document(params.text_document.uri)
69         ls.parse(doc)
70
71         for uri, (version, diagnostics) in ls.diagnostics.items():
72             ls.text_document_publish_diagnostics(

```

```

73         types.PublishDiagnosticsParams(
74             uri=uri,
75             version=version,
76             diagnostics=diagnostics,
77         )
78     )
79
80
81 if __name__ == "__main__":
82     logging.basicConfig(level=logging.INFO, format="%(message)s")
83     start_server(server)
84

```

É obviamente um LSP muito básico, e a maneira como iteramos pelo documento (linha a linha) impede que este sistema consiga detetar blocos multi-linha. É útil apenas para ficheiros prolog de factos e regras, um por linha.

```

1 p(a).
2 r(a)
3 q(a) :- p(a).

```

Figura 5: Falta de um ponto final na segunda linha é detetado.

Gramática Independente de Contexto

Uma ferramenta mais poderosa para este tipo de aplicação seria uma gramática independente de contexto (CFG). Se conseguirmos alimentar o texto completo do documento a uma gramática que reconheça Prolog, e que reporte erros com expressão suficiente para o nosso caso de uso, teremos construído um LSP interessante.

Para a gramática de Prolog o professor apontou-nos para o Tree Sitter de Prolog¹⁵, que define por completo a CFG do Prolog.

Para construir este sistema encontrámos o PLY^{16,17}, uma implementação em Python das ferramentas lex¹⁸ e Yacc¹⁹. O que é relevante porque o nosso grupo usou ferramentas

¹⁵<https://github.com/DataGrout/tree-sitter-grammars/blob/main/tree-sitter-prolog/grammar.js>

¹⁶<https://github.com/dabeaz/ply>

¹⁷<https://ply.readthedocs.io/en/latest/ply.html>

¹⁸[https://en.wikipedia.org/wiki/Lex_\(software\)](https://en.wikipedia.org/wiki/Lex_(software))

¹⁹<https://en.wikipedia.org/wiki/Yacc>

²⁰[https://ftp.gnu.org/old-gnu/Manuals/flex-2.5.4/html_mono/flex.html]

²¹https://www.gnu.org/software/bison/manual/html_node/index.html#SEC_Contents

análogas (Flex²⁰ e GNU Bison²¹) para construir um compilador em C²², na Unidade Curricular de Compiladores no primeiro semestre.

Usando a nossa (pouca) experiência em GNU Bison desse trabalho, construímos uma gramática simples (a notação é a que é usada pelo Bison, uma adaptação de Backus–Naur form²³):

bnf

CFG simples

```
1 source_file
2   : clauses
3   ;
4
5 clauses
6   : clauses clause
7   | clause
8   ;
9
10 clause
11  : fact
12  | rule
13  ;
14
15 fact
16  : ATOM "(" arglist ")" "."
17  ;
18
19 rule
20  : ATOM "(" arglist ")" ":-" body "."
21  ;
22
23 body
24  : ATOM "(" arglist ")"
25  | ATOM "(" arglist ")" "," body
26  ;
27
28 arglist
29  : term
30  | term "," arglist
31  ;
32
33 term
34  : ATOM
35  | VARIABLE
36  | NUMBER
37  | ATOM "(" arglist ")"
38  ;
```

²²<https://github.com/Eurico-M/compiladores/tree/main>

²³https://en.wikipedia.org/wiki/Backus%E2%80%93Naur_form

Nesta gramática, um ficheiro é uma lista de cláusulas (definida recursivamente). Uma cláusula é um facto ou uma regra, que são definidos seguidamente. Seguindo a notação de Prolog, átomos e variáveis são diferentes (átomos começam por letras minúsculas, variáveis por letras maiúsculas).

Para implementar esta gramática, precisamos primeiro de criar tokens a partir do nosso ficheiro de texto. Para isso usamos a parte Lex do PLY:

python

simple-cfg.py

```
1 import ply.lex as lex
2
3 tokens = (
4     "ATOM",
5     "VARIABLE",
6     "NUMBER",
7     "COMMA",
8     "LPAREN",
9     "RPAREN",
10    "IMPLIES",
11    "PERIOD",
12 )
13
14 t_COMMA = r","
15 t_LPAREN = r"\"
16 t_RPAREN = r"\"
17 t_IMPLIES = r":-\"
18 t_PERIOD = r"\"
19 t_VARIABLE = r"[A-Z_][a-zA-Z0-9_]*"
20 t_ATOM = r"[a-z][a-zA-Z0-9_]*"
21 t_NUMBER = r"[0-9]+"
22
23 t_ignore = " \t\r"
24 t_ignore_COMMENT = r"%[^\n]*"
25
26
27 def t_newline(t):
28     r"\n+"
29     t.lexer.lineno += len(t.value)
30
31
32 def t_error(t):
33     raise SyntaxError(f"Illegal character '{t.value[0]}' at line
34 {t.lineno}")
35
36 lexer = lex.lex()
```

Definimos os nossos tokens, as expressões regulares que os definem, e funções auxiliares para contar números de linha e lidar com caracteres desconhecidos.

Com o tokenizador construído, podemos fazer um parser ao estilo do Yacc/Bison:

python

simple-cfg.py

```
1 import ply.yacc as yacc
2
3 def p_source_file(p):
4     """source_file : clauses
5     | empty"""
6     if len(p) == 2:
7         p[0] = p[1]
8
9     No comentário da função temos a definição da regra gramatical, e depois
temos uma notação inspirada pelo Yacc, $$ = $1, ou seja, o source_file é o
resultado de uma lista de clauses.
10
11
12 def p_clauses(p):
13     """clauses : clauses clause
14     | clause"""
15     if len(p) == 3:
16         p[0] = p[1] + [p[2]]
17     else:
18         p[0] = [p[1]]
19
20 def p_clause(p):
21     """clause : fact
22     | rule"""
23     p[0] = p[1]
24
25 def p_fact(p):
26     """fact : ATOM LPAREN arglist RPAREN PERIOD"""
27     p[0] = ("fact", p[1], p[3], p.lineno(1))
28
29     Aqui definimos valores que são guardados no nó da AST.
30
31
32 def p_rule(p):
33     """rule : ATOM LPAREN arglist RPAREN IMPLIES body PERIOD"""
34     p[0] = ("rule", p[1], p[3], p[6], p.lineno(1))
35
36
37 def p_body(p):
38     """body : ATOM LPAREN arglist RPAREN
39     | ATOM LPAREN arglist RPAREN COMMA body"""
40     if len(p) == 5:
41         p[0] = [(p[1], p[3])]
42     else:
```

```
41     p[0] = [(p[1], p[3])] + p[6]
```

```
42
```

```
43
```

```
44 def p_arglist(p):
```

```
45     """arglist : term
```

```
46     | term COMMA arglist"""
```

```
47     if len(p) == 2:
```

```
48         p[0] = [p[1]]
```

```
49     else:
```

```
50         p[0] = [p[1]] + p[3]
```

```
51
```

```
52
```

```
53 def p_term(p):
```

```
54     """term : ATOM
```

```
55     | VARIABLE
```

```
56     | NUMBER
```

```
57     | ATOM LPAREN arglist RPAREN"""
```

```
58     if len(p) == 2:
```

```
59         p[0] = p[1]
```

```
60     else:
```

```
61         p[0] = (p[1], p[3])
```

```
62
```

```
63
```

```
64 def p_empty(p):
```

```
65     """empty : """
```

```
66     p[0] = []
```

```
67
```

```
68
```

```
69 def p_error(p):
```

```
70     if p:
```

```
71         raise SyntaxError(
```

```
72             f"Syntax error at '{p.value}' (line {p.lineno})",
```

```
73             ("error", p.lineno, 0, "error"),
```

```
74         )
```

Aproveitamos o `SyntaxError` do Python para guardar, para além da string de erro, informação do número de linha (e, eventualmente, coluna, que para já é 0).

```
75
```

```
76
```

```
77 parser = yacc.yacc()
```

Repare-se que, como é expectável de um parser, podemos construir uma Abstract Syntax Tree. Assim, esta informação pode ser usada pelo nosso LSP para todas as funções mais complexas (análise semântica, *go-to-definition*, encontrar referências, etc.).

Estas funções não foram ainda implementadas no ficheiro `exemplo simple-cfg.py`. De facto, a nossa função `parse()` é bastante simples:

python

simple-cfg.py

```
1 def parse(self, document: TextDocument):
2     diagnostics = []
3
4     try:
5         lexer.lineno = 1
6
7         result = parser.parse(document.source, lexer=lexer)
8         0 resultado do parser é a AST, e poderia ser usado para várias funções do
9         LSP. Aqui está simplesmente a ser ignorado (e imprimido directamente).
10
11         if DEBUG:
12             import sys
13
14             print("--- AST ---", file=sys.stderr)
15             print(result, file=sys.stderr)
16             print("--- END OF AST ---", file=sys.stderr)
17
18         Podemos ver o resultado da impressão no buffer das mensagens de EGLOT no
19         Emacs.
20
21     except SyntaxError as e:
22
23         if e.lineno is not None:
24             error_line = e.lineno - 1
25         else:
26             error_line = 0
27
28         line_length = len(document.lines[error_line])
29
30         diagnostics.append(
31             types.Diagnostic(
32                 message=str(e),
33                 severity=types.DiagnosticSeverity.Error,
34                 range=types.Range(
35                     start=types.Position(line=error_line, character=0),
36                     end=types.Position(line=error_line,
37                                     character=line_length),
38                 ),
39             )
40         )
41
42     self.diagnostics[document.uri] = (document.version, diagnostics)
```

É uma função muito simples que serve apenas de exemplo, e carece de vários melhoramentos. Por exemplo, só assinalamos a linha onde o erro é reportado, não calculamos a coluna. Mas o mais crucial, o parser só lida com um erro. Se houver dois erros

no ficheiro, só lidámos com o primeiro que for apanhado pelo parser. Para isso, teríamos que acrescentar recuperação de erros ao nosso parser.

Podemos, para já, ver a AST que é construída pelo PLY.

```
1 p(a).  
2  
3 q(a) :- p(a).
```

Figura 6: Simple base de dados Prolog com facto e regra.

```
[stderr] --- AST ---  
[stderr] [('fact', 'p', ['a'], 1), ('rule', 'q', ['a'], [('p', ['a'])], 3)]  
[stderr] --- END OF AST ---
```

Figura 7: AST imprimida no buffer do EGLLOT.

Podemos ver que a nossa AST é constituída por um fact, p, que tem como argumentos a, na linha 1, e uma rule, q, com argumentos a e cujo corpo é uma lista, neste caso unitária, p com argumentos a, na linha 3.

Global

A pedido do professor, criámos um pequeno programa, `global.pl`.

Mantemos os predicados `validate_file/2`, `validate_source/3`, `validate_text/3`, `term_expansion/2`, `analyze/5`, `rule/5`, `directive/4`, `body/5`, do ficheiro `lsp.yap`.

Podemos lançar este programa com o YAP, e usando como exemplo `file.pl`:

bash

```
yap
```

YAP

```
?- [global].
?- validate_file('./file.pl', E).
```

Para verificar os `def` e `use` na base de dados:

YAP

```
?- listing(def/6).
?- listing(use/9).
```

Para criar imagens de grafos, podemos criar um ficheiro DOT²⁴, para mostrar os predicados usados por outros predicados:

prolog

global.pl

```
1 write_dot :-
2     open('graph.dot', write, Out),
3     writeln(Out, 'digraph Calls {}'),
4
5     forall(use(Called, CalledArity, _, Caller, CallerArity, _, Line, _,
6     _),
7         format(Out, '    "~w/~w" -> "~w/~w:~w";~n', [Caller, CallerArity,
8         Called, CalledArity, Line])),
9     writeln(Out, '}'),
10    close(Out).
```

E usando Graphviz²⁵:

bash

```
dot -Tpng graph.dot -o graph.png
```

Obtemos o seguinte grafo:

²⁴[https://en.wikipedia.org/wiki/DOT_\(graph_description_language\)](https://en.wikipedia.org/wiki/DOT_(graph_description_language))

²⁵<https://graphviz.org/doc/info/command.html>

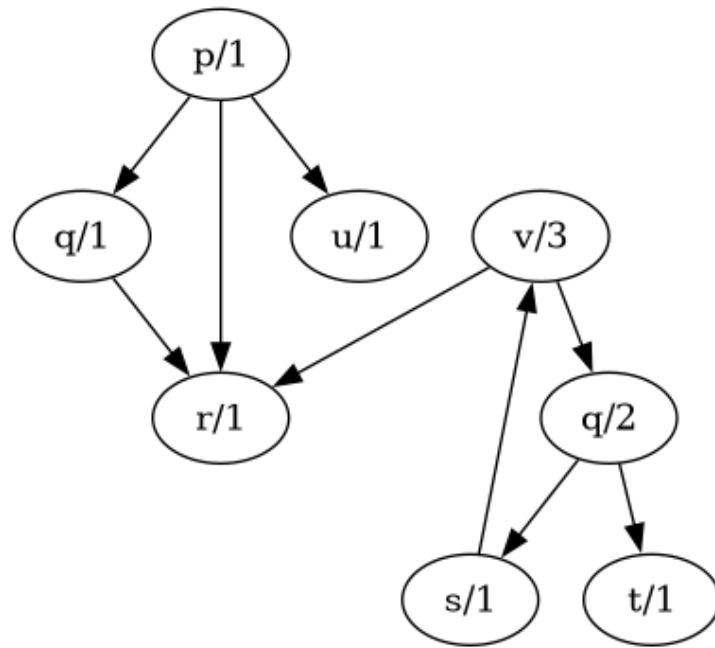


Figura 8: Grafo de chamadas.

Que corresponde a file.pl:

prolog

file.pl

```

1 p(a).
2 p(X) :-
3     q(X),
4     r(X).
5 p(X) :-
6     u(X).
7
8 q(X) :-
9     r(X).
10 q(X, Y) :-
11     s(X),
12     t(Y).
13
14 r(a).
15 r(b).
16
17 s(a).
18 s(b).
19 s(c).
20
21 u(d).
22
23 v(X, Y, Z) :-
24     r(X);
25     q(Y, Z).

```

Os seguintes predicados foram acrescentados e são úteis à análise do documento prolog:

list_calls/1

Podemos listar todos os use, que podemos pensar como se fossem as arestas do grafo que ligam os predicados:

prolog

global.pl

```
1 list_calls(L) :-  
2     findall((Caller/CallerArity, Called/CalledArity: Line),  
3         use(Called, CalledArity, _, Caller, CallerArity, _, Line, _, _),  
4         L).
```

list_by_caller/2

Se estivermos interessados num único predicado que chama outros:

prolog

global.pl

```
1 list_by_caller(Caller, L) :-  
2     findall((Caller/CallerArity, Called/CalledArity: Line),  
3         use(Called, CalledArity, _, Caller, CallerArity, _, Line, _, _),  
4         L).
```

list_by_called/2

Ou um predicado que é chamado por outros:

prolog

global.pl

```
1 list_by_called(Called, L) :-  
2     findall((Caller/CallerArity, Called/CalledArity: Line),  
3         use(Called, CalledArity, _, Caller, CallerArity, _, Line, _, _),  
4         L).
```

list_by_arity/2

Para listar todos os predicados com uma determinada aridade:

prolog

global.pl

```
1 list_by_arity(Arity, L) :-  
2     findall(Name/Arity: Line, def(Name, Arity, _, Line, _, _), L).
```

entry_points/1

Se um predicado é definido, mas nunca é usado, é um ponto de entrada:

prolog

global.pl

```
1 entry_points(L) :-  
2     findall(Name/Arity:Line,  
3         (  
4             def(Name, Arity, _, Line, _, _),  
5             \+ use(Name, Arity, _, _, _, _, _, _)
```

```
6      ),
7      L).
```

undefined_calls/1

De uma maneira semelhante, podemos listar os predicados que são usados mas nunca foram definidos:

prolog

global.pl

```
1 undefined_calls(L) :-
2     findall(Name/Arity:Line,
3         (
4             use(Name, Arity, _, _, _, _, Line, _, _),
5             \+ def(Name, Arity, _, _, _, _)
6         ),
7     L).
```

edge/2

Para preservar a sanidade que nos resta, pensemos no predicado use como uma aresta do grafo de chamadas, que liga dois nós (predicados). Este predicado é útil para os próximos predicados baseados em grafos:

prolog

global.pl

```
1 edge(A/Aar, B/Bar) :-
2     use(B, Bar, _, A, Aar, _, _, _, _).
```

connected/3

Dados dois predicados, podemos verificar se estão “ligados”, ou seja, se, transitivamente, o primeiro chama o segundo. A lista devolvida representa o caminho da chamada. Para evitar ciclos precisamos de manter uma lista de predicados visitados.

prolog

global.pl

```
1 connected(Start, End, Path) :-
2     connected(Start, End, Path, [Start]).
3
4 connected(Start, End, [Start, End], _Visited) :-
5     edge(Start, End).
6
7 connected(Start, End, [Start|Rest], Visited) :-
8     edge(Start, Middle),
9     \+ member(Middle, Visited),
10    connected(Middle, End, Rest, [Middle|Visited]).
```

reachable_from/2

Com o predicado `connected` já construído, é fácil agora fixar o nó onde começamos a procura e iterar por todos os nós ligados a esse. No fim ordenamos a lista para apresentar apenas um conjunto ordenado.

prolog

global.pl

```
1 reachable_from(Start, L) :-  
2     findall(End, connected(Start, End, _), Unsorted),  
3     sort(Unsorted, L).
```

cycles/1

Como podemos pensar nos predicados como se fossem um grafo de chamadas dirigido, usámos o algoritmo de três cores (branco, cinzento, preto) para deteção de ciclos em grafos²⁶. Nesta variante, não marcámos os nós com cores, mas mantemos duas listas que servem o mesmo propósito.

prolog

global.pl

```
1 cycles(L) :-  
2     findall(Name/Arity, cycle_path(Name/Arity, [], []), L).  
3  
4 cycle_path(Node, Visited, RecStack) :-  
5     member(Node, RecStack),  
6     !.  
7  
8 cycle_path(Node, Visited, RecStack) :-  
9     member(Node, Visited),  
10    !,  
11    fail.  
12  
13 cycle_path(Node, Visited, RecStack) :-  
14     edge(Node, Next),  
15     cycle_path(Next, [Node|Visited], [Node|RecStack]).
```

²⁶<https://www.geeksforgeeks.org/dsa/detect-cycle-in-a-graph/>

Conclusão

O trabalho realizado foi singular, na medida em que envolveu o estudo e compreensão de vários componentes e a sua interação. De certo modo, fez-nos lembrar o trabalho de desenvolver um compilador na Unidade Curricular de Compiladores.

Apesar de não ser um trabalho desenvolvido totalmente em Prolog, ganhámos uma compreensão desta linguagem, através do estudo da sua Gramática Independente de Contexto, e principalmente da análise do ficheiro `lsp.yap`.

Os vários componentes do projeto requereram o uso de Prolog, Python, JSON, TypeScript (numa fase inicial de integração com o VSCode), Lisp (quando Emacs substituiu VSCode), o que para os membros mais generalistas do grupo foi um aspeto positivo e enriquecedor (ainda que por vezes complexo).

Sem a ajuda do professor, a nossa intuição seria seguir o caminho descrito no capítulo Alternativas: construir um parser da gramática de Prolog, capaz de criar uma Abstract Syntax Tree útil para as funcionalidades do LSP, e capaz de reportar erros de uma maneira robusta suficiente para uso no LSP.

É possível que também tivéssemos ficado um pouco perdidos na complexidade de implementar uma extensão de Visual Studio Code, que estudámos inicialmente, e cujo *debugging* se começava a revelar moroso. Assim, outra vantagem deste trabalho foi o facto de nos expôr ao Emacs. Por ser simples de personalizar (através da sua configuração escrita em Lisp) e com o seu sistema de *buffers* que expõe várias mensagens úteis, percebemos porque é que este programa ainda é usado atualmente.

Temos assim várias avenidas de exploração para o futuro: projectos em Prolog, em particular alguns dos outros trabalhos mencionados pelo professor nas aulas (interrogação de bases de dados e jogos são dois projetos que considerámos como alternativa); extensões de Visual Studio Code; LSP vs Tree Sitter, e outros sistemas de análise sintática e semântica de código fonte, e a sua integração com editores de texto e IDEs; o uso de Emacs; entre outras.